

Daniel Toledo-Navarro

Professor Burge

Project 3 Code Review Summary

Overview

This project had us review each other's Boggle solvers the way a team would on GitHub. I started with my solver from Assignment 2, which stores the dictionary in a trie and searches the grid with depth-first search and backtracking. I opened pull request #14 with the original code, and my partner, Aja Wilkes, reviewed it. I then reviewed her solver in pull request #23. My original version is saved as `boggle_solver.py.orig`.

Feedback I Received

Aja liked the trie, the backtracking with one shared visited set, and the way the solver handles "Qu" tiles. Her suggestions were about readability. The single-letter names in `find_words` (w, v and t) were hard for her to follow, and she noticed that my method names mixed camelCase and snake_case.

Feedback I Gave

Aja's solver stops searching a path once it is no longer a dictionary prefix, and I told her that was a good choice. I also pointed out that several of her methods used two-space indentation while the rest of the file used four, which PEP 8 and the linter would flag. When I ran her code, a grid with uneven rows crashed with an index error, so I suggested a validity check before the search starts. I had first guessed that an empty grid would crash too, but testing showed it does not, so I corrected my comment before submitting it.

Improvements I Implemented

I took both of Aja's suggestions. The parameters in `find_words` are now `prefix`, `visited` and `node`, and the helper methods are `validate_grid`, `validate_dict` and `build_trie`. I left `getSolution` alone because the tests call it by that name. While I was in the file, I also added docstrings, replaced the magic numbers with named constants, and wrapped the long lines.

Static Analysis Results

I ran pycodestyle. My solver had two lines over 79 characters, and Codio also flagged a missing newline at the end of the file. That one took a couple of tries, because the editor kept adding spaces to the last line. The bigger surprise was tests.py, where the autograder reported more than 40 problems. Most were tabs mixed with spaces and two-space indentation, along with missing blank lines, semicolons and comment formatting. I cleaned up both files without changing what the tests check, and both now pass with no warnings.

Regression Testing

After each change I re-ran the checks. All 26 autograder tests pass, and the final solver gives the same output as the original on the sample grid and on eight extra cases, including empty grids, uneven rows and a “Qu” tile.

Reflection

The most useful part was seeing how my code looked to someone else. I knew what w, v and t stood for because I wrote them, but Aja had no way to know. Reviewing her code also showed me that running it finds more than reading it. The uneven-row crash only appeared when I tried it, and my first guess about empty grids was wrong. I also had not realized that code can pass every test and still fail the linter. On a team, I would keep phrasing comments as questions and start with what works, since that made Aja’s feedback easy to accept.